

AI SOFTWARE COMPANY SIMULATOR

An AI-Powered Platform for End-to-End Software Development Planning and Management

Tagline

Experience how a real software company builds software using AI.

AWT / Advanced Web Technology Project

Team Members

Ayushi Shah

Manav Shukla

Ziya Shukla

Saniya Saiyed

Department: Computer Science & Engineering

GitHub Repository

github.com/24bt04157-star/AI-Software-Company

1. Project Overview

AI Software Company Simulator is an AI-powered web platform designed to simulate the workflow of a real software development company. Instead of functioning as a general-purpose chatbot, the platform uses specialized AI roles to move a software idea through structured stages of the Software Development Life Cycle (SDLC).

A user enters a software idea. The system can then perform requirement gathering, generate an SRS, create user stories, propose architecture and database design, plan sprints, estimate cost and timeline, prepare development guidance, generate testing artifacts, and present the project through a centralized dashboard.

Core differentiator: the platform is workflow-driven and organized around specialized AI employees rather than a single conversational assistant.

2. Problem Statement

Many students, entrepreneurs, startups and freelancers have software ideas but may not know how to translate those ideas into professional software requirements, architecture, planning and testing artifacts.

- Difficulty identifying complete requirements.
- Limited understanding of professional software-development workflows.
- Difficulty estimating project timeline and cost.
- Difficulty preparing software documentation such as SRS and UML.
- Need for multiple disconnected planning and tracking tools.

3. Proposed Solution

The proposed system acts as an AI software company. Specialized AI employees are assigned responsibilities corresponding to roles found in a software organization. Their outputs form a connected workflow, allowing the user's initial idea to become a structured development plan.

4. Objectives

- Simulate a real software-company workflow.
- Automate important SDLC stages.
- Reduce manual project-planning effort.
- Generate professional software-engineering documents.
- Help students understand software-development processes.
- Assist startups and project owners with structured planning.
- Provide a centralized project dashboard.

5. Scope

- Requirement collection and analysis
- SRS generation
- User-story generation
- UML and ER diagram planning/generation
- Database design
- API planning

- Sprint planning
- Timeline and cost estimation
- Task assignment
- Bug/test planning
- Project-progress dashboard

6. AI Employee Architecture

The proposed architecture divides responsibilities among specialized AI roles. Each role has a defined purpose and produces artifacts for subsequent stages.

AI Role	Main Responsibility	Example Outputs
Business Analyst	Gather and clarify requirements	Requirements, user needs, functional/non-functional requirements
Project Manager	Plan and organize the project	Sprints, milestones, timeline, tasks, cost estimate
Solution Architect	Design the technical solution	Architecture, database, APIs, UML, ER design
UI/UX Designer	Plan the user experience	Wireframes, screen flow, UI suggestions, color palette
Frontend Developer	Plan client-side implementation	Components, modules, folder structure
Backend Developer	Plan server-side implementation	APIs, authentication, database integration
QA Engineer	Plan quality assurance	Test cases, bug reports, edge cases, checklist

7. End-to-End Workflow

Login → Enter Software Idea → Business Analyst → Requirement Gathering → SRS → Project Manager → Sprint Planning → Solution Architect → Database/UML → UI Designer → Wireframes → Frontend/Backend Planning → QA Planning → Dashboard → Project Tracking

8. Major Functional Modules

8.1 Authentication

- User registration
- User login
- JWT-based authentication

8.2 Project Management

- Create project
- Edit project
- Delete project
- Store project information

8.3 Requirement Gathering

- AI interview/questions
- Requirement collection
- Functional requirements
- Non-functional requirements

8.4 Documentation

- SRS
- User stories
- UML/use-case artifacts

- ER diagram
- Database schema
- API list

8.5 Planning

- Sprint planning
- Timeline
- Milestones
- Task assignment
- Budget/cost estimation

8.6 Design

- Wireframes
- Screen flow
- Color palette
- UI suggestions
- System architecture

8.7 Development Planning

- Technology-stack recommendations
- Frontend architecture
- Backend architecture
- Modules
- CRUD operations

8.8 QA

- Test cases
- Bug reports
- Edge cases
- Quality checklist

8.9 Dashboard

- Overall progress
- Completed tasks
- Pending tasks
- Timeline
- Budget
- Documents

9. Proposed User Experience

The platform should begin with a guided onboarding experience. The supplied VenturePilot reference is being used as a UX reference for a guided onboarding approach: the user answers key questions before entering the main project workspace.

- Choose project/application type.
- Enter project name.
- Describe the software idea.
- Identify target users.
- Specify important features.
- Identify requirements such as login, payments, mobile app and admin panel.
- Confirm the generated project brief.
- Enter the AI software-company dashboard.

10. Main Screens / Pages

Screen	Purpose
Landing Page	Explain the product and start a project.
Login / Signup	Authenticate users.
Onboarding	Understand the user's project idea and requirements.
Project Creation	Create and configure a software project.
AI Company / Agents	Show specialized AI roles and progress.
Requirements	Display gathered and generated requirements.
SRS	Display/download generated SRS.
Architecture	Display technical architecture, database and API planning.
UML / ER	Display software design artifacts.
UI/UX	Display wireframe and screen recommendations.
Sprint / Tasks	Show milestones, tasks and sprint planning.
QA	Show test cases, bugs and quality checks.
Dashboard	Show overall project progress and key project information.

11. Technology Stack

Layer	Technology
Frontend	HTML, CSS, Bootstrap, JavaScript
Backend	Node.js, Express.js
Database	MongoDB
Authentication	JWT
AI	Gemini API / OpenAI API

Layer	Technology
Charts	Chart.js
Deployment	Vercel / Netlify

12. Suggested Technical Architecture

Client Layer: HTML/CSS/Bootstrap/JavaScript interface communicates with backend REST APIs.

Application Layer: Node.js and Express.js manage authentication, projects, tasks, AI requests and business logic.

AI Orchestration Layer: AI prompts/agents are organized by role, with outputs from earlier stages passed as structured context to later roles.

Data Layer: MongoDB stores users, projects, requirements, generated artifacts, tasks, sprints and QA information.

Authentication: JWT-based authentication protects user/project access.

Visualization: Chart.js can be used for progress and project metrics.

13. Suggested Database Collections

Collection	Purpose / Example Data
users	User accounts and roles.
projects	Project name, idea, users, status, progress and dates.
requirements	Functional/non-functional requirements and clarifications.
documents	SRS, user stories, UML/ER metadata and generated content.
tasks	Task title, description, AI role/assignee, status and priority.
sprints	Sprint name, goals, dates and tasks.
bugs	Bug title, severity, status, description and test context.
ai_outputs	Structured outputs generated by AI agents, where needed.

14. Functional Requirements

- The system shall allow users to register and log in.
- The system shall allow authenticated users to create and manage projects.
- The system shall collect a software idea and project information.
- The system shall support AI-assisted requirement gathering.
- The system shall generate structured project documentation.
- The system shall generate or present architecture/design artifacts.
- The system shall create sprint and task plans.
- The system shall support progress tracking.
- The system shall display project information in a centralized dashboard.
- The system shall support testing/QA planning artifacts.

15. Non-Functional Requirements

- Responsive user interface.
- Secure authentication.
- Clear navigation and understandable workflow.
- Reasonable AI response time.
- Scalable application architecture.
- Reliable storage of project data.
- Maintainable modular codebase.

16. Example Project Journey

Example input: “I want to create a platform where students can find internships and companies can post internship opportunities.”

Business Analyst: identifies user groups and derives functional requirements.

Project Manager: converts requirements into milestones and sprints.

Solution Architect: proposes architecture, database entities and APIs.

UI/UX Designer: proposes screens such as login, student profile, company dashboard, internship listing and application pages.

Developer roles: produce implementation structure and module/API planning.

QA: creates test cases for registration, search, filtering and application workflows.

Dashboard: brings project status and artifacts together.

17. Project Advantages

- One integrated platform for multiple software-planning stages.
- AI-powered workflow automation.
- Educational value for students learning SDLC.
- Structured documentation generation.
- Centralized project visibility.

- Potential usefulness for startups and project owners.
- Clear demonstration value for an AWT project.

18. Existing System / Comparison Context

The project proposal identifies general AI assistants such as ChatGPT, Claude and Gemini, and project-management/documentation tools such as Notion, Jira and Trello as existing solutions. The proposed project's distinction is its attempt to combine a software-company workflow with specialized AI roles and project artifacts inside one application.

19. Future Scope

- Automatic code generation.
- GitHub integration.
- Figma integration.
- Cloud deployment.
- AI coding agents.
- DevOps automation.
- Live collaboration.
- Mobile application.
- Voice-based AI project manager.

20. Development Plan

Phase	Work
1. Repository & Setup	Clean GitHub structure, README, frontend/backend/docs folders and environment setup.
2. UI Prototype	Landing page, onboarding, project creation and dashboard.
3. Authentication	Signup, login and protected routes/API access.
4. Backend & Database	Express APIs, MongoDB models and project data.
5. Requirements Agent	AI-assisted project interview and requirement generation.
6. Documentation	SRS, user stories and structured artifacts.
7. Architecture	Database/API/UML/ER planning.
8. Planning	Sprints, tasks, milestones, timeline and cost.
9. QA	Test cases, bugs and quality checklist.
10. Integration	Connect frontend, backend, database and AI.
11. Testing	Functional, UI, API and integration testing.
12. Finalization	Deployment, screenshots, report, presentation and GitHub cleanup.

21. GitHub Repository Structure

AI-Software-Company/

■■■ README.md

■■■ frontend/

■■■ backend/

■■■ docs/

The repository should be updated incrementally. Each major feature should be committed separately so the project history demonstrates development progress to the faculty reviewer.

Example commit sequence:

- Initial project structure
- Added landing page
- Added onboarding flow
- Added project dashboard
- Added requirement analysis
- Added SRS generation
- Added database
- Connected backend APIs
- Added AI agent
- Added testing and bug tracking

22. Team Responsibilities

The project team consists of the following four members. The exact division of responsibilities can be finalized according to the work actually completed by each member.

Member	Possible Responsibility
Ayushi Shah	Project lead, frontend integration and overall coordination
Manav Shukla	Backend/API development
Ziya Shukla	AI/agent integration and documentation
Saniya Saiyed	Database, QA/testing and UI support

23. Testing Strategy

- Unit testing for key frontend/backend functions.
- API testing for authentication, projects and AI-related endpoints.
- Database testing for create/read/update/delete operations.
- UI testing across major screens and responsive layouts.
- Integration testing across frontend, backend, database and AI services.
- AI-output validation for required fields and expected document structure.
- User-acceptance testing using sample software ideas.

24. Expected Outputs

- Software Requirement Specification (SRS)
- User Stories
- Use Case / UML artifacts
- ER Diagram
- Sprint Plan
- Timeline
- Cost Estimation
- API List
- Database Schema
- Project Dashboard
- QA/Test artifacts

25. Future Product Direction

The academic version demonstrates AI-assisted software-project planning. A future product version could extend the same workflow into automatic code generation, GitHub integration, Figma integration, cloud deployment, DevOps automation, live collaboration and voice-based project management.

26. Conclusion

AI Software Company Simulator introduces a structured approach to software project planning by combining multiple specialized AI roles into one collaborative platform. Unlike a single conversational interface, the proposed system follows a software-development workflow and produces structured artifacts at each stage.

By automating requirement gathering, project planning, documentation, architecture planning, development guidance, testing planning and project monitoring, the platform demonstrates how AI can be applied to a realistic software-engineering workflow. The project is designed as an educational AWT application and as a foundation that could be expanded into a larger software-planning product.

27. Suggested Final Report Structure

- Cover Page
- Certificate
- Acknowledgement
- Abstract
- Table of Contents
- List of Figures
- Chapter 1 – Introduction
- Chapter 2 – Literature Survey
- Chapter 3 – Problem Statement
- Chapter 4 – Proposed System
- Chapter 5 – System Design (UML, ER Diagram, Architecture)
- Chapter 6 – Implementation
- Chapter 7 – Testing
- Chapter 8 – Results & Screenshots
- Chapter 9 – Future Scope
- Chapter 10 – Conclusion
- References
- Appendix

28. Project Reference Notes

This document consolidates the supplied AWT project brief, including the project title, objectives, AI roles, workflow, modules, technology stack, expected outputs, future scope and report structure. It is intended as a working master document for development, project review and preparation of the final academic report.